

AyuSphere · Technology & Security Guide

How the system is built — every technology, workflow, and safeguard

Problem statement **SIH26046** · Ministry of Ayush · All India Institute of Ayurveda

Cloud-based Clinical Trial Management System for Ayurveda — real-time, role-based, GCP-aligned.

Live system: <https://ayusphere-three.vercel.app>

Companion documents: 7 role UAT guides · Judge Demo Script (8-minute storyline)

Generated 14 September 2026. Versions, the permission matrix and signature statements in this document are extracted from the codebase at generation time — the document cannot drift from the product.

1 · What we built

AyuSphere is a working, deployed clinical trial management system for AIIA's Ayurveda trials — not a mock-up. It covers the full trial lifecycle (draft → IEC review → CTRI registration → active → completed → locked), participant screening/consent/enrolment with permuted-block randomization, template-driven e-CRFs with electronically signed approvals, a complete pharmacovigilance loop (MedDRA/WHODrug-coded AEs, configurable escalation deadlines, CIOMS-style reports, safety-signal detection, DSMB packs, NPvCC spontaneous-ADR intake), monitoring visits and data queries, protocol amendments under IEC control, and standards-based interoperability (CDISC SDTM/ADaM/Define-XML and a live two-way FHIR R4 API).

Seven enforced roles each see a tailored workspace; every mutation is validated, permission-checked and audited in the same database transaction; and the whole system is gated by **297 automated tests** running against real Postgres.

2 · Technology stack — what and how

Frontend

Technology	Version	What it is	How AyuSphere uses it
Next.js (App Router, Turbopack)	16.3.4	Full-stack React framework — server components, server actions, API routes in one codebase.	Every screen is a server component that reads the session and renders only what the role may see; mutations are server actions, so clinical data logic never ships to the browser. One repo, one deploy.
React	19.2.8	UI library.	Server components for data-heavy pages; small client islands only where interactivity needs it (forms, Sphera panel, toasts, mobile drawer).

TypeScript	5.x	Typed JavaScript.	Strict mode across app, services, tests and scripts — `pnpm typecheck` is part of the merge gate. Database row types are inferred from the schema, so a schema change breaks the build, not production.
Tailwind CSS	4.x	Utility-first CSS.	Implements the design system (glassmorphism + claymorphism tokens, one font, two text sizes). A static test suite (design-qa) fails the build if a screen violates the design rules.
framer-motion	13.2.0	Animation library.	A small approved motion catalog (page fade, card hover, orb pulse) — decorative only, never load-bearing.
Recharts	3.10.1	Charting library.	Dashboard KPIs: enrolment progress, AE severity mix, portfolio charts on the regulator/leadership dashboards.
SWR	2.5.1	Client-side data fetching with revalidation.	Live unread badges and toasts — the sidebar counts refresh on navigation and on a `refreshBadges` event instead of being frozen into the server layout.
lucide-react · clsx · CVA · tailwind-merge · date-fns	—	Icons and small utilities.	Consistent iconography, conditional class composition, and date arithmetic for deadline clocks.

Backend & data

Technology	Version	What it is	How AyuSphere uses it
------------	---------	------------	-----------------------

Neon Postgres (serverless)	driver 1.1.0	Managed serverless Postgres with WebSocket connections (the `ws` package supplies the socket in Node).	The single system of record: 25+ tables covering trials, sites, participants, CRFs, AEs, signatures, amendments, queries, audit events, guide threads. Scales to zero between demos.
Drizzle ORM + drizzle-kit	0.45.2	Type-safe SQL query builder and migration generator.	Every query is built from the typed schema and executed as a parameterized statement — string-concatenated SQL does not exist in the codebase. Schema changes flow through generated, versioned migrations (0001–0009).
Zod	4.6.2	Runtime schema validation.	Every service input has a Zod schema — types, lengths, enums, ranges are enforced at the boundary before any database work. The e-CRF field ranges the PI sees are the same schemas.
Vercel Blob	2.8.0	Object storage.	Consent forms, uploaded documents and doctor-note images, stored under unguessable UUID paths and referenced from the database.

Auth & security

Technology	Version	What it is	How AyuSphere uses it
Auth.js (NextAuth v5)	5.0.0-beta.32	Authentication framework.	Credentials sign-in with a JWT session capped at 12 hours; the token carries the user's role and site, so every request knows who is acting. CSRF protection is built in.
bcryptjs	3.0.3	Adaptive password hashing.	Passwords hashed at cost factor 12 — and re-verified a second time whenever a user electronically signs a record.
node:crypto (SHA-256)	Node 24 built-in	Cryptographic hashing.	Each e-signature stores the SHA-256 hash of the canonical JSON of the signed record — later tampering is detectable because the hash no longer matches.

AI

Technology	Version	What it is	How AyuSphere uses it
LangGraph + LangChain	1.4.15 / 1.2.11	Agent-graph framework for LLM applications.	Sphera (the in-app guide) is a LangGraph StateGraph: it loads the user's persisted thread, injects only THAT role's knowledge, streams tokens to the client, and saves the turn back to Postgres — memory survives refreshes and devices.
Groq (ChatGroq)	1.3.1	Low-latency LLM inference API.	Powers Sphera's replies. The admin pastes the API key once in Settings; it is stored server-side and serves every role — with an environment-variable fallback for local dev.

Anthropic Claude SDK	0.125.0	Claude API client.	The clinical copilot and the doctor-note extraction flow (image → structured draft CRF, always human-approved before it becomes data).
----------------------	---------	--------------------	--

Testing & tooling

Technology	Version	What it is	How AyuSphere uses it
Vitest + PGLite	5.0.0 / 0.5.8	Test runner + Postgres compiled to WebAssembly.	297 tests in 47 files run against a REAL embedded Postgres — RBAC denials, audit atomicity, signature refusal, randomization balance and export formats are all tested with genuine SQL semantics, not mocks.
Playwright	1.63.0	Browser automation.	Live end-to-end sweeps against the deployed site (login flows, badges, tabs, mobile drawer) and the PDF generators that produced this document and the UAT guides.
Faker	10.6.0	Synthetic data generation.	Every participant, CRF value and AE in the demo database is synthetic — no real patient data anywhere (see Privacy).
ESLint 9 · tsx · pnpm	pnpm@9.15.9	Lint, TypeScript script runner, package manager.	`pnpm verify` = typecheck + full test suite; nothing merges red.

Platform

Technology	Version	What it is	How AyuSphere uses it
Vercel	—	Cloud hosting for Next.js.	Auto-deploys every push to main; Vercel Cron drives the alert/deadline sweeps (protected by a bearer secret). Fluid compute — no servers to manage, meets the PS's cloud-based requirement.
GitHub (trunk-based)	—	Version control & collaboration.	Single main branch, small verified commits; the repository itself is an audit trail of how the system was built.

3 · Architecture & workflows

3.1 · The life of every write (the core workflow)

Every clinical-record change in AyuSphere — enrolment, CRF entry, AE capture, approval — travels the same pipeline. There is no second path around it:

Browser — server-rendered screen; the role sees only permitted actions

HTTPS · Auth.js CSRF protection

Next.js server action / API route — session read from the 12-hour JWT

invalid → rejected with field errors

Zod schema — shape, types, lengths, ranges validated; bad input never reaches logic

denied → RbacError, nothing executes

RBAC assertCan(role, capability) — permission re-checked in the service, not just the UI

Service layer — business rules (state machines, deadline rules, randomization)

an unaudited write cannot commit

withAudit() transaction — the mutation AND its audit row commit together or not at all

Drizzle ORM → Neon Postgres — parameterized SQL only; audit_events is insert-only

3.2 · Sphera — the AI guide with memory

User asks Sphera (leaf orb, any screen)

per-user thread — no cross-user leakage

/api/guide — session checked; the user's thread loaded from `guide_messages`

the guide cannot describe screens the role cannot open

LangGraph StateGraph — injects ONLY this role's knowledge base (click paths, permissions)

key never sent to the browser

ChatGroq — key from admin Settings (server-side), env fallback; 20s timeout

memory survives refresh, devices, sessions

Streamed tokens → browser · turn persisted back to Postgres

3.3 · Interoperability — data out, data in

Out: one click exports CDISC **SDTM** (DM/AE domains), **ADaM ADSL**, **Define-XML** (real variable-level metadata), and a **FHIR R4 Bundle**; a live API (`GET /api/fhir/Bundle/[trialId]`) serves `application/fhir+json` to any authorized system. **In:** inbound FHIR Observation bundles are parsed and land as **draft** CRF entries only — an imported value becomes clinical data exclusively after a human with `crf.enter` permission reviews it and an approver signs. Nothing auto-commits.

4 · Privacy vs Security — and why a CTMS needs both

Security = protecting the data

Keeping systems and data safe from unauthorized access, tampering and loss — whoever the attacker is.

- Answers: "Can the wrong person get in, read, or change this?"
- Tools: authentication, authorization, hashing, validation, audit trails, transport encryption.
- Failure looks like: a breached account, a forged record, an injected query.

Privacy = governing the data

Rules about what personal data is collected at all, why, who may see it, and for how long — even from insiders.

- Answers: "Should this data exist here, and who is entitled to see it?"
- Tools: data minimization, de-identification, consent, purpose limitation, role-scoped visibility.
- Failure looks like: a perfectly "secure" system that stores names it never needed.

The relationship: security protects whatever data you hold; privacy decides what you hold and who is entitled to it. You can have security without privacy (a well-locked vault full of data you should never have collected) — but you can never have privacy without security. AyuSphere is designed for both: the strongest privacy control in the system is that **identifying data is never collected in the first place** (§6), and everything that IS collected sits behind the security layers in §5. This is the posture India's **DPDP Act** and the ICMR/GCP guidelines expect of clinical systems.

5 · Security in AyuSphere — every layer, as implemented

5.1 · Authentication — who you are

- Credentials sign-in via **Auth.js v5**; passwords hashed with **bcrypt**, **cost factor 12** (~250ms per attempt — brute force is computationally expensive by design). Plaintext passwords are never stored or logged.
- Sessions are signed **JWTs capped at 12 hours** carrying the user's id, role and site; inactive (deactivated) accounts are refused at sign-in. CSRF protection is built into Auth.js.

5.2 · Authorization — what you may do (RBAC)

A single capability matrix (`src/lib/rbac.ts`) is the only source of permission truth. The UI hides what you cannot do, but the **service layer independently re-checks** every call with `assertCan()` — bypassing the UI changes nothing.

Capability	PI	Coord	Monitor	Ethics	PV	Admin	Regulator
View dashboard	●	●	●	●	●	●	●
Manage trials & lifecycle 🗑️	●	●				●	
IEC review & amendments 🗑️				●		●	
Manage sites 🗑️	●					●	
Screen / consent / enrol 🗑️	●	●				●	

Enter CRF data 	•	•				•	
Approve CRFs (signed) 	•					•	
Capture adverse events 	•	•			•	•	
PV review & report (signed) 					•	•	
Log monitoring visits 				•		•	
Raise / close data queries 				•		•	
Acknowledge alerts 	•	•	•		•	•	
Use AI copilot	•	•			•	•	
Internal messaging	•	•	•	•	•	•	•
Run exports (CDISC/FHIR)	•				•	•	
Upload documents 	•	•				•	
Manage users 						•	
Browse audit trail						•	•

• = capability granted ·  = a mutating capability (changes clinical records). Rendered directly from `src/lib/rbac.ts` at generation time — this table cannot disagree with the running app. Note the Regulator column: no  capability is granted, an invariant enforced by an automated test.

5.3 · Auditability — who did what (ALCOA+)

- Every mutation runs inside `withAudit()` : the change and its audit event are inserted **in the same database transaction**. An unaudited write cannot commit; a failed write leaves no audit noise.
- Each audit row stores the actor's id and role, the action, the entity, and **before/after JSON snapshots** — the table is insert-only. Even system-initiated writes (cron deadline sweeps) are attributed to a fixed system actor, so *every* row is attributable.
- The regulator role browses this trail read-only, including e-signature evidence.

5.4 · Electronic signatures — record-freezing actions

- Approving a CRF, approving a correction, and marking a safety report submitted all require the actor to **re-enter their password**, which is bcrypt-verified against their own active account — a stolen open session cannot sign.
- The signature row stores a **SHA-256 hash of the canonical JSON** of the signed record (keys recursively sorted, so the hash is order-independent) and commits **in the same transaction** as the guarded mutation: no signature, no change; refused signature, nothing written. Any later tampering with the record is detectable — its hash no longer matches.
- Each signature carries a fixed, 21-CFR-11-style meaning statement, shown verbatim at signing:
 - **approve**: *"I approve this record as accurate and complete."*
 - **correct**: *"I approve this correction as accurate and complete; the original record remains preserved."*
 - **report**: *"I confirm this safety report is complete and has been submitted to the regulatory authority."*

5.5 · Input safety

- **Validation**: every service input is parsed by a Zod schema before any logic runs — types, lengths, enums, numeric ranges. Malformed input is rejected at the boundary with field-level errors.
- **SQL injection**: structurally prevented — all database access goes through Drizzle ORM, which emits parameterized statements only. There is no string-built SQL anywhere in the codebase.

- **XSS:** React escapes rendered output by default; the app renders no untrusted raw HTML.

5.6 · Secrets & service protection

- The Groq API key is entered once by the admin in Settings and stored **server-side**; the settings page receives only a boolean "key is set" — **the key value is never sent to any browser**. Environment variables serve as fallback; the `.env` file is never committed.
- The scheduled alert sweep (`/api/cron/alerts`) requires a **bearer secret** (`CRON_SECRET`) — outsiders cannot trigger or spam the alert engine.
- All traffic is HTTPS (TLS) end to end: browser → Vercel → Neon.

5.7 · Safe interoperability

- FHIR import creates **drafts only** behind the `crf.enter` capability; imported data becomes clinical record only through the same signed human approval as manually entered data.
- AI outputs follow the same rule: the doctor-note extraction produces a draft a human must approve — no model output ever writes itself into a clinical record.

Hardening roadmap (stated honestly): this is a competition prototype and three upgrades are planned for production: (1) uploaded documents currently use unguessable-URL public blob storage → move to private, access-checked blob reads; (2) add a strict Content-Security-Policy and related security headers; (3) encrypt the stored AI API key at rest with a KMS-managed key. None of these affect clinical data, which lives entirely in Postgres behind the layers above.

6 · Privacy in AyuSphere — DPDP-aligned by design

- **De-identification is structural, not procedural.** The participants table has **no name, phone, address or date-of-birth columns** — **they do not exist in the schema**. A participant's only identity is a subject code like `AYU-001-P-0042`. The system cannot leak what it cannot store; re-identification keys stay offline at the site, exactly as GCP source documents intend.
- **Data minimization:** only trial-relevant fields (eligibility, arm, consent state, visit data) are collected.
- **Consent as data:** consent is recorded against the **specific consent-form version** signed; publishing a new form version automatically flags affected participants for **re-consent**. Withdrawal is a first-class, audited state.
- **Role-scoped visibility:** the RBAC matrix (§5.2) is also a privacy control — an ethics member or monitor sees only what their duty requires (purpose limitation, enforced in code).
- **Demo data policy:** the live system carries **real CTRI registry trials** (public records) but **100% synthetic participants** generated with Faker — no real patient data exists anywhere in the deployment.
- **Accountability:** the same audit trail that serves GCP serves DPDP — every touch of participant data is attributable and reviewable.

7 · What makes this build stand out

An unaudited write cannot commit. Audit isn't a log file bolted on — it's a transactional invariant. If the audit insert fails, the clinical change rolls back with it. Most systems log after the fact; AyuSphere cannot forget.

Signatures are cryptographic, not cosmetic. Password re-verification + SHA-256 record hash + fixed meaning statement, atomic with the change it covers — a checkbox UI can be faked; this cannot.

One knowledge base powers the AI guide, the UAT documents — and this document. Sphera's role knowledge, the 7 UAT guides, the RBAC table in §5.2 and the version numbers in §2 are all generated from the same source files, with drift-guard tests that fail if documentation and product ever disagree. Zero documentation rot, by construction.

297 tests on real Postgres. PGlite embeds actual Postgres in the test runner — RBAC denials, audit atomicity, signature refusals, randomization balance and export formats are proven against genuine SQL semantics on every commit.

Interoperability is live, both directions. Judges can curl the FHIR endpoint during the demo; Define-XML carries real variable-level metadata; inbound bundles flow through the same human-signed approval as manual entry.

Regulation is configurable, not hard-coded. AE/SAE escalation deadlines, enrolment-lag thresholds and monitoring cadence live in an admin-edited (and audited) rule table — when NDCT timelines change, no code changes.

Scientific rigor built in: permuted-block randomization at enrolment, monitor data queries that block approval until answered, protocol amendments gated by the IEC, DSMB packs from live disproportionality signals, and the NPvCC national-pharmacovigilance intake AIIA actually operates.

AI with guardrails and memory. Sphera remembers you across sessions (Postgres-persisted LangGraph threads) but knows only your role's screens — and no AI output anywhere becomes clinical data without a signed human approval.

8 · Engineering workflow

- **Trunk-based development:** one `main` branch, small frequent commits, every push auto-deploys to Vercel.
- **Merge gate:** `pnpm verify` = strict TypeScript typecheck + the full 297-test suite; nothing ships red.
- **Schema discipline:** every database change is a generated, versioned Drizzle migration applied to Neon — the schema's history is replayable.
- **Live E2E:** Playwright sweeps run against the deployed site itself (logins for all 7 roles, badges, tabs, search, mobile drawer) — we test what judges will actually touch.
- **Docs as code:** architecture decisions (28 ADRs), the gap analysis against the problem statement, task tracking and these generated guides all live in the repository.